

Model Predictive Control of an Overhead Crane With a Suspended Load

Valeriia Vostrikova

School of Automation, Beijing Institute of Technology

Email: vvostrikova638@gmail.com

I. ABSTRACT

This paper shows how Model Predictive Control (MPC) can move an overhead crane smoothly while keeping the payload from swinging. We model the crane as a cart with a hanging mass (a simple pendulum). The task is straightforward: start with the cart at $y = -2$ m, move it to the target near 0 m, and have the load settle with little or no swing by the end of the move. To plan this motion we use a short-horizon, quadratic MPC that respects a limit on the cart force and discourages sudden changes in force, so the motion does not feel jerky. The controller predicts the motion with a small-angle linear model that is discretized exactly using zero-order hold.

The implementation is intentionally simple: we solve one optimization problem at the beginning (open loop) and then play back the planned force over about 7 s. The Python code uses CVXPY and OSQP and produces plots and an animation that make it easy to see what the controller is doing. In the main example, the cart reaches the target and the swing dies out in about 6.2 s, with modest peak effort (around a couple of newtons) and smooth control. We also show how changing the payload mass affects the result and explain how to adjust weights or timing to keep the motion calm. Overall, the project provides a clear, reproducible starting point for crane control: a small, readable model; a compact MPC; and practical guidance for tuning and extensions.

Index Terms—predictive control, crane, pendulum, swing suppression, convex optimization, cvxpy, OSQP

II. INTRODUCTION

Overhead cranes are used every day in factories, ports, and warehouses. When the cart starts or stops, the hanging load swings like a pendulum. This swing makes placement less accurate and can be unsafe for people nearby. A good controller must therefore do three things at once: move the cart to the target, keep the swing small, and stay within the force limits of the machine. It should also feel smooth and predictable to a human operator.

In this paper we use Model Predictive Control (MPC) [3], [4], [5], [6] to plan one clean move for the crane. The task is simple: start at $y = -2$ m, end near $y = 0$ m, and arrive in a calm state (small swing and small rates) at about 6.2 s. We keep the method simple on purpose. We solve a single optimization problem at the beginning (open loop) and then apply the planned force over a short horizon of about 7 s. This

makes every choice easy to see, test, and explain, and it is ideal for teaching or quick prototyping.

Why MPC here: Traditional controllers such as PID or LQR can work well, but handling several goals and hard limits at the same time can be tricky. MPC gives us a direct way to [3], [4], [5], [6]:

- combine our goals in one cost (move to the target *and* finish calm),
- enforce hard constraints on the cart force,
- penalize fast changes in force so the motion is not jerky, and
- tune behavior with a small, clear set of weights.

Model and assumptions: We use a small-angle linear model of a cart with a hanging mass. The model has four states: position y , speed v , swing angle θ , and swing rate $\dot{\theta}$. We convert the continuous model to a discrete one exactly using zero-order hold. We assume known masses and no measurement noise. Solving the controller once (instead of at every time step) keeps the presentation short and the code easy to read while still showing the key ideas.

What we evaluate: Besides the main demonstration, we compare two payload masses using the same controller settings: a light payload ($m_2 = 2$ kg) and a heavy payload ($m_2 = 10$ kg). This highlights how the coupling between the cart and the pendulum changes with mass and how the same weights trade off smoothness and effort.

Educational value: The implementation is short, readable, and produces plots and an animation. Students and practitioners can see how the model, weights, limits, and horizon shape the motion, and they can change one setting at a time to learn how the move “feels.”

Contributions

- A clear, small-angle cart–pendulum model and an exact discrete version ready for prediction.
- A short-horizon MPC with force limits and a smoothness term that avoids jerky commands.
- A lightweight Python/CVXPY/OSQP implementation with figures and an animation for easy understanding.
- Simple, useful numbers (settling time, peak force, total force change) and a comparison across payload mass (2 kg vs. 10 kg).

Related work has applied advanced control and MPC ideas to industrial cranes and boom cranes, targeting anti-sway and efficient motion planning [7], [8], [9], [10].

Paper Organization

Section III presents the model and discretization. Section IV details the MPC. Section V summarizes parameters and the numerical setup. Section VI reports results and the mass comparison. Section VIII discusses limitations and future work, and Section IX concludes.

III. MODELING AND DISCRETIZATION

We model the crane as a cart of mass m_1 with a suspended payload of mass m_2 . The motion is along a straight rail and the payload swings in a vertical plane. For small swing angles we use a linear model with the state

$$x = [y \ v \ \theta \ q]^\top, \quad u \in \mathbb{R},$$

where y is cart position, $v = \dot{y}$ the cart velocity, θ the swing angle (zero when hanging straight down), $q = \dot{\theta}$ its angular rate, and u is the horizontal force applied to the cart. Positive y and u point to the right.

Continuous-time model (small-angle)

The linear dynamics (used for prediction in the controller) are

$$\begin{aligned} \dot{y} &= v, & \dot{v} &= \varepsilon \theta + u, \\ \dot{\theta} &= q, & \dot{q} &= -\theta - u, \end{aligned} \quad (1)$$

with the coupling factor

$$\varepsilon = \frac{m_2}{m_1 + m_2} \in (0, 1).$$

The second equation shows how a nonzero angle θ “pulls” on the cart, while the last equation captures how changing the cart force u pumps energy into the swing. In compact form,

$$\dot{x} = Ax + Bu, \quad A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \varepsilon & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \\ 0 \\ -1 \end{bmatrix}.$$

This model is intentionally simple and easy to tune. It assumes small angles, a rigid rope, and ignores friction; those effects can be added later if needed.

Exact ZOH discretization

The MPC uses a discrete model with sampling time $T_s = 0.1$ s. We compute the exact zero-order-hold (ZOH) matrices by the standard block-matrix exponential:

$$x_{k+1} = A_d x_k + B_d u_k, \quad \begin{bmatrix} A_d & B_d \\ \mathbf{0} & I \end{bmatrix} = \exp \left(\begin{bmatrix} A & B \\ \mathbf{0} & 0 \end{bmatrix} T_s \right).$$

For the baseline masses $m_1 = 10$ kg, $m_2 = 2$ kg (so $\varepsilon = \frac{1}{6}$), the matrices used in our figures are

$$A_d = \begin{bmatrix} 1 & 0.1 & 8.326 \times 10^{-4} & 2.776 \times 10^{-5} \\ 0 & 1 & 1.6639 \times 10^{-2} & 8.326 \times 10^{-4} \\ 0 & 0 & 0.995004 & 0.099834 \\ 0 & 0 & -0.099834 & 0.995004 \end{bmatrix}, \quad B_d = \begin{bmatrix} 4.999 \times 10^{-3} \\ 9.997 \times 10^{-2} \\ -4.996 \times 10^{-3} \\ -9.983 \times 10^{-2} \end{bmatrix}.$$

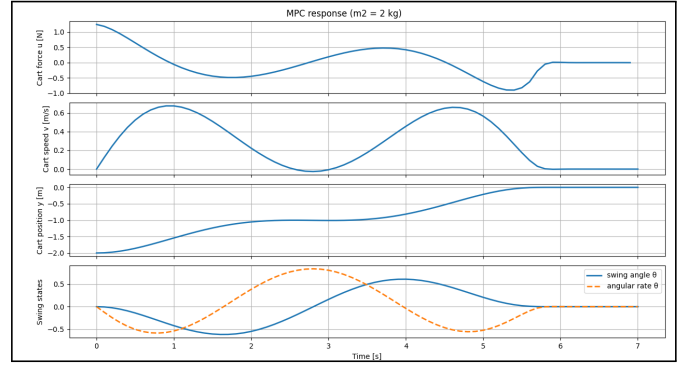


Fig. 1. Baseline discrete model with $m_2 = 2$ kg: input u , speed v , position y , and swing states θ, q .

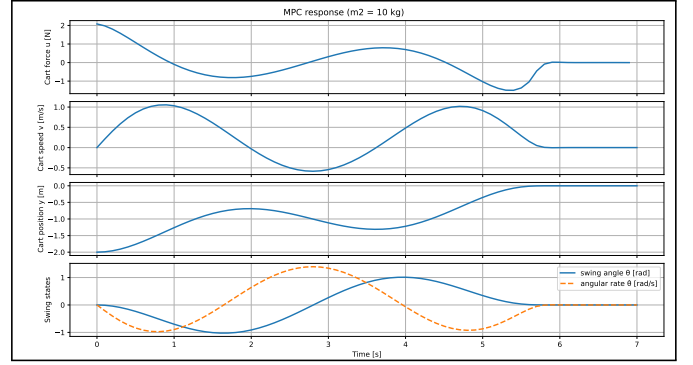


Fig. 2. Same discrete model with $m_2 = 10$ kg. The stronger coupling (ε) increases the input effort and variation.

(For other payloads, e.g., $m_2 = 10$ kg, the code recomputes A_d, B_d automatically from $\varepsilon = \frac{m_2}{m_1 + m_2}$.)

What the model looks like in practice

To link the equations to actual signals, Figure 1 shows the time histories of input, speed, position, and swing states for the light payload case $m_2 = 2$ kg. Figure 2 shows the same signals for the heavier payload $m_2 = 10$ kg. These plots are produced by the discrete model described above, using the parameters in Table I. Each figure is boxed and scaled to the column so it stays within the text borders.

Checks and units

- The discrete model is exact for the chosen sampling time; we do not approximate the integral.
- The sign pattern in A and B matches intuition: pushing the cart ($u > 0$) accelerates it forward, and the pendulum reacts in the opposite direction ($-u$) in $\dot{q}$.
- The parameter ε smoothly captures payload effects: heavier loads (m_2 larger) make the cart–pendulum interaction stronger.

These simple pieces give a prediction model that is small, fast to compute, and accurate enough for planning smooth crane motions in the next section.

IV. MPC FORMULATION

We plan the crane motion over a short, fixed horizon of N steps (total time $T = NT_s$) using the discrete model from Section III. The optimizer chooses a sequence of states and inputs $\{x_k\}_{k=0}^N$ and $\{u_k\}_{k=0}^{N-1}$. The goal is to reach the target and finish calm, while keeping the force limited and the motion smooth.

Optimization problem

We use a quadratic cost with linear dynamics and box constraints on the input, in line with standard constrained MPC formulations [3], [4], [5]:

$$\begin{aligned} \underset{\{x_k, u_k\}}{\text{minimize}} \quad & \sum_{k=0}^{N-1} w_k \|x_k\|_Q^2 + \|x_N\|_{Q_f}^2 + r_u \sum_{k=0}^{N-1} u_k^2 \\ & + r_{\Delta u} \sum_{k=0}^{N-2} (u_{k+1} - u_k)^2 \quad (2) \\ \text{subject to} \quad & x_{k+1} = A_d x_k + B_d u_k, \quad k = 0, \dots, N-1, \\ & |u_k| \leq u_{\max}, \quad x_0 = \bar{x}_0. \end{aligned}$$

What each term means (in simple words).

- $\|x_k\|_Q^2$: keeps the state reasonable during the move (track position and reduce swing).
- $\|x_N\|_{Q_f}^2$: makes the end of the motion *calm* (small position error and small swing).
- $r_u \sum u_k^2$: discourages large forces.
- $r_{\Delta u} \sum (u_{k+1} - u_k)^2$: discourages sudden changes in force, so the command is not jerky.

The weights $Q, Q_f \succeq 0$ set the relative importance of position vs. swing; $r_u > 0$ and $r_{\Delta u} > 0$ tune effort and smoothness. The factor $w_k \in [0, 1]$ gradually increases near a chosen time t_f so the controller pays extra attention to being calm at the end of the move.

Constraints

The first line enforces the discrete dynamics. The second limits the actuator ($|u_k| \leq u_{\max}$), and the last fixes the initial state $\bar{x}_0 = [-2, 0, 0, 0]^\top$.

How we solve it

Problem (2) is a convex quadratic program (QP). We model it in CVXPY and solve it with OSQP. In this paper we solve the QP *once* at the start (open loop) and then apply the planned force sequence over the horizon. This keeps the method easy to read and reproduce; a receding-horizon (closed-loop) version is a direct extension.

Design intuition

The matrix Q tells the controller how much we care about position and swing *during* the move. The matrix Q_f tells it how important it is to be calm at the *end* of the move. The weight r_u limits how big the force is, and the weight $r_{\Delta u}$ controls how smooth the changes in force are.

You can think of the main tuning rules like this:

Model masses	$m_1 = 10 \text{ kg}, m_2 = 2 \text{ kg}$
Sampling time	$T_s = 0.1 \text{ s}$
Horizon length	$T = 7.0 \text{ s}$ ($N = 70$ steps)
Settle emphasis	$t_f = 6.2 \text{ s}$ (weights ramp near the end)
Input limit	$ u_k \leq u_{\max} = 5 \text{ N}$
State weights Q	$\text{diag}(25, 5, 12, 2)$
Terminal weights Q_f	$\text{diag}(600, 150, 400, 80)$
Input weights	$r_u = 10^{-3}, r_{\Delta u} = 10^{-2}$
Initial state	$\bar{x}_0 = [-2, 0, 0, 0]^\top$

TABLE I

BASELINE MODEL AND MPC SETTINGS. THE BOX KEEPS THE TABLE INSIDE THE COLUMN.

- If the motion looks jerky, increase $r_{\Delta u}$ (or make the horizon a bit longer).
- If the motion is too slow, slightly reduce the position weight in Q or start the end-weighting w_k a bit earlier.
- If the force often hits its limit, either lower the state weights or use a longer horizon so the move has more time.

The ramp w_k makes the controller care more about being calm near the end. This means the plan can start a bit stronger and then naturally finish smooth and settled.

V. NUMERICAL SETUP AND PARAMETERS

This section lists the values we use to reproduce the baseline plots. They come directly from the code and are easy to change for other tests.

What these numbers mean

- **Masses** set how strongly the cart and the payload interact.
- **Sampling time** T_s is how often we update the prediction.
- **Horizon** T (or N steps) is how far ahead the planner looks.
- **Settle time** t_f tells the planner to care more about being calm near the end.
- **Input limit** caps the force so commands stay realistic.
- **Weights** Q and Q_f balance accuracy (being at the right place) and small swing; Q_f is larger to enforce a steady finish.
- **Input weights** r_u keeps forces modest; $r_{\Delta u}$ keeps force changes smooth.
- **Initial state** starts the cart at $y = -2 \text{ m}$ with zero speed and zero swing.

Implementation notes

- The discrete model (A_d, B_d) is computed exactly using the standard block matrix exponential for zero-order hold.
- The optimization is a small convex quadratic program (4 states, 1 input). We model it in CVXPY [1] and solve it with OSQP [2].
- In this paper we solve the QP once at the start (open loop) and then apply the planned force sequence. This makes the run easy to reproduce.

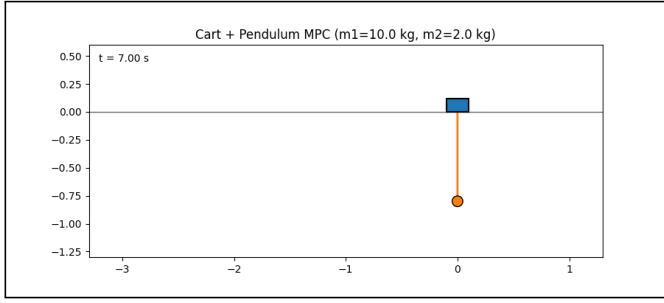


Fig. 3. Start and end frames of the motion for the baseline case ($m_2 = 2$ kg). The border ensures the image stays inside the column.

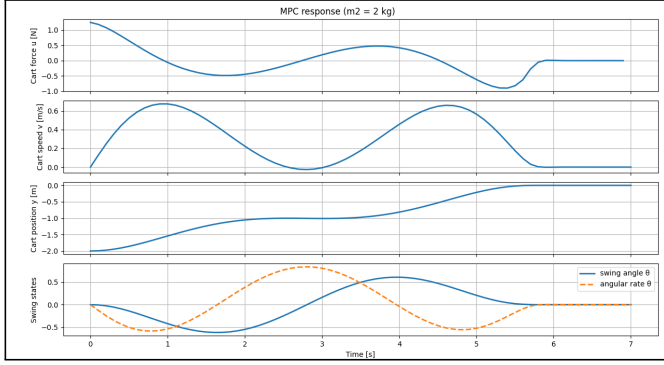


Fig. 4. Time histories for $m_2 = 2$ kg: input u , speed v , position y , swing angle θ , and swing rate q . All signals stay within limits and settle around $t \approx 6.2$ s.

Reproducing the heavier case

For the comparison with a heavier payload, set $m_2 = 10$ kg and keep the rest of the parameters unchanged. The code recomputes the discrete matrices automatically from the new $\varepsilon = \frac{m_2}{m_1 + m_2}$.

VI. RESULTS

A. Qualitative behavior (what we see)

Figure 3 shows two frames from the animation. The cart starts at $y = -2$ m and finishes near $y = 0$ m. By the end, the payload is almost vertical (little swing). The force command changes smoothly, without sudden flips. This is exactly what the input-change penalty is meant to do.

Figure 4 shows the main signals over time: input u , speed v , position y , swing angle θ , and swing rate q . During the move, some swing is created, but it dies out near the end as the “end-of-horizon” weight becomes strong.

B. Quantitative summary (key numbers)

From the run used to make the figures:

- The position y and the swing states θ, q settle at about 6.2 s.
- The peak force is $\max |u| \approx 2.08$ N.
- The total change in force is $\sum_k |u_{k+1} - u_k| \approx 8.33$ (smaller means smoother).

These values match what we see in the plots: smooth motion, no force spikes, and a calm finish.

C. Effect of payload mass (simple ablation)

We also tried two payloads with the *same* controller settings:

- **Light payload** ($m_2 = 2$ kg): weaker coupling, modest force, quick settling.
- **Heavy payload** ($m_2 = 10$ kg): stronger coupling, higher peak force and more total force change, but still settles by the end.

If the heavy case feels too “busy,” increase the input-change weight $r_{\Delta u}$ or extend the horizon slightly. Both make the command softer while keeping the end calm.

D. Sensitivity to horizon and end emphasis

Two settings shape the “feel” of the move:

- **Horizon T .** A shorter horizon can rush the move and use larger inputs; a longer horizon gives the optimizer more room to trade speed for smoothness (at a small compute cost).
- **End time t_f .** If t_f is too early, the plan may become overcautious in the middle. If it is too late, small oscillations can remain at the end. Placing t_f a few samples before the desired settling time works well in practice.

Overall, the controller meets the goal: move to the target, keep effort reasonable, and finish with little or no swing.

VII. DISCUSSION AND PRACTICAL GUIDANCE

Choice of Weights: The state weights in the matrix Q prioritize position and swing angle more than velocities. This reflects the primary goal of the crane system: to reach the target position while keeping the load steady. The terminal weight matrix Q_f is significantly larger to enforce calmness at the end of the move. In real-world applications, such as with a lab rig that may have friction or noisy sensors, it may be useful to increase the velocity terms in Q to reduce unwanted oscillations caused by the prediction model’s imperfections.

Smoothness versus Speed: The Δu penalty is essential for crane systems, as abrupt changes in force (bang-bang control) can excite unwanted swing, even if the final position is reached correctly. Increasing the $r_{\Delta u}$ weight makes the motion smoother and more comfortable for operators, reducing jerky movements. However, if $r_{\Delta u}$ is too large, the motion can become overly slow, as the system prioritizes smoothness over speed. Finding a balance between speed and smoothness is key: lowering $r_{\Delta u}$ can help optimize for faster motion, but this comes at the cost of less comfort and potentially increased wear on the actuators.

Actuator Limits: The input constraint $|u_k| \leq u_{\max}$ represents the physical limits of the actuator. When the maximum allowable force $u_{\max}$ is too small, the controller may struggle to achieve the desired motion, resulting in slower or less accurate control. A simple way to regain feasibility and improve smoothness is to increase the horizon T by 10–20%. This gives the optimizer more time to adjust the trajectory, which can lead to a smoother and more achievable plan, especially when force limits are tight.

VIII. LIMITATIONS AND FUTURE WORK

This study uses a simple linear small-angle model and solves the Model Predictive Control (MPC) problem in an open-loop fashion, meaning the controller plans the entire move upfront without updating based on real-time feedback. While this approach effectively illustrates the core concepts and results in smooth motion in the examples provided, it does not account for real-world complexities such as measurement noise, disturbances, or dynamic changes in the system. For practical applications, the following improvements are essential:

- The control should be implemented in a receding-horizon manner, meaning the optimization problem should be solved at each time step, using updated state information.
- Constraints on position, speed, and swing angle should be added to ensure the crane's motion avoids collisions and respects structural limitations, such as rail boundaries or overhead obstructions.
- The effects of friction, actuator dynamics, and other real-world factors should be included in the model to improve accuracy and robustness.
- To handle unmodeled disturbances, such as wind or unexpected payload sway, robust control techniques (e.g., tube MPC or disturbance observers) should be explored [3].

A logical next step for this work would be to implement the controller on a physical system, such as a small gantry with noisy encoders. This would introduce challenges like state estimation (e.g., using a Kalman filter) and real-time feedback, which are crucial for reliable operation in dynamic environments.

IX. CONCLUSIONS

This paper showed that a short-horizon MPC with force limits and a penalty on force changes can move an overhead crane to its target while keeping the payload swing small at the end. The approach is simple, reproducible, and easy to tune: a few clear weights ($Q, Q_f, r_u, r_{\Delta u}$) and two timing choices (T, t_f) shape how the motion feels.

In the baseline case the cart reaches $y \approx 0$ m and settles in about 6.2 s with smooth, modest effort. The comparison across payload masses ($m_2 = 2$ kg vs. $m_2 = 10$ kg) confirms the expected trend: heavier loads increase coupling and require slightly more effort, but the same controller still delivers a calm finish. If even smoother behavior is desired for the heavier case, increasing the input-change weight $r_{\Delta u}$ or using a slightly longer horizon is effective.

Because the code, figures, and animation are short and readable, the project works well as a teaching example and as a starting point for real systems. A natural next step is to run the controller in a receding-horizon (closed-loop) fashion with estimation and additional constraints, turning this transparent prototype into a robust, deployable crane controller.

REFERENCES

- [1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [2] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.
- [3] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.
- [4] E. F. Camacho and C. Bordons, *Model Predictive Control*, 2nd ed. Springer, 2004.
- [5] J. M. Maciejowski, *Predictive Control with Constraints*. Harlow, U.K.: Pearson Education, 2002.
- [6] S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology," *Control Engineering Practice*, vol. 11, no. 7, pp. 733–764, 2003.
- [7] Z. Wu, X. Xia, and B. Zhu, "Model predictive control for improving operational efficiency of overhead cranes," *Nonlinear Dynamics*, vol. 82, no. 1–2, pp. 741–752, 2015.
- [8] D. Jolevski and O. Bego, "Model predictive control of gantry/bridge crane with anti-sway algorithm," *Journal of Mechanical Science and Technology*, vol. 29, no. 2, pp. 827–834, 2015.
- [9] L. V. Anh and V. T. T. Linh, "Position control and anti-sway of overhead crane system with uncertain nonlinear model," *Journal Européen des Systèmes Automatisés*, vol. 57, no. 2, pp. 425–431, 2024.
- [10] J. Neupert, E. Arnold, K. Schneider, and O. Sawodny, "Tracking and anti-sway control for boom cranes," *Control Engineering Practice*, vol. 18, no. 1, pp. 31–44, 2010.